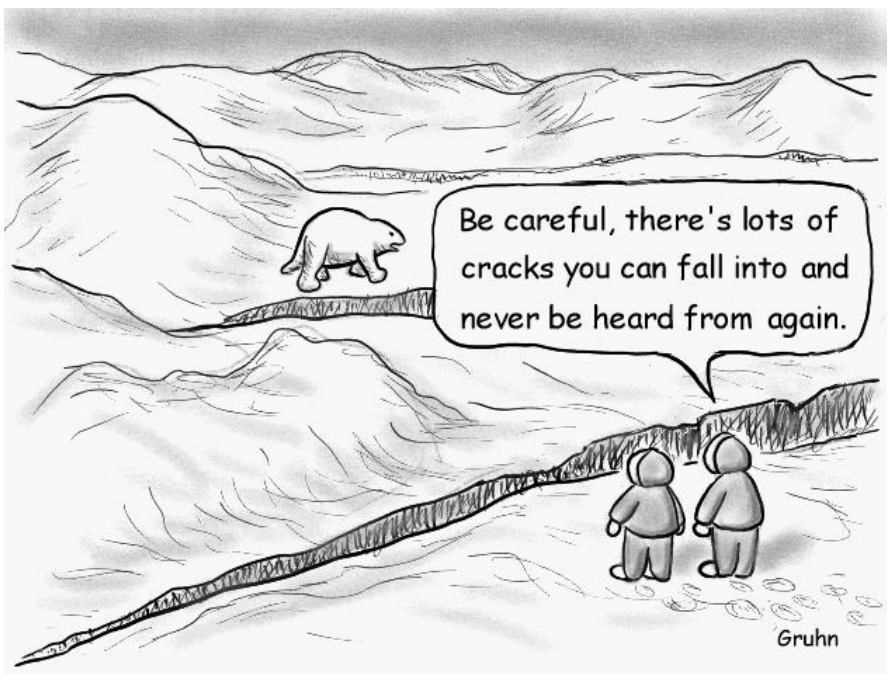
2

DESIGN LIFECYCLE

Chapter Highlights

2.1 Hindsight/Foresight
2.2 Findings of the HSE
2.3 Design Lifecycle
- 2.3.1 Hazard & Risk Analysis
- 2.3.2 Allocation of Safety Functions to Protective Layers
- 2.3.3 Develop Safety Requirements Specification
- 2.3.4 SIS Design & Engineering
- 2.3.5 Installation, Commissioning, and Validation
- 2.3.6 Operations and Maintenance
- 2.3.7 Modifications
- 2.3.8 Decommissioning

Summary
References



"If I had 8 hours to cut down a tree, I'd spend 6 hours sharpening the axe."

— A. Lincoln

Designing a single component may be a relatively simple matter, one that a single person can handle. Designing any large *system* however, whether it's a car, a computer, or an airplane, is typically beyond the ability of any single individual. The instrument or control system engineer should *not* feel that all the tasks associated with designing a safety instrumented system are his or her responsibility alone, because they're not. The design of a system, including a safety instrumented system, requires a multi-discipline *team*.

2.1 Hindsight/Foresight

"Hindsight can be valuable when it leads to new foresight."

— P. G. Neumann

Hindsight is easy. Everyone always has 20/20 hindsight. *Fore*sight, however, is a bit more difficult. Foresight is required, however, with today's large, high risk systems. We simply can't afford to design large petrochemical plants by trial and error. The risks are too great to learn that way. We have to try and prevent certain accidents, no matter how remote the possibility, even if they have never yet happened. This is the subject of *system safety*.

System safety was born out of the military and aerospace industries. The military have many obvious high risk examples. The following case may have been written in a lighthearted fashion, but was obviously a very serious matter to the personnel involved. Luckily, there were no injuries.

> *An ICBM silo was destroyed because the counterweights, used to balance the silo elevator on the way up and down, were designed with consideration only to raising a fueled missile to the surface for firing. There was no consideration that, when you were not firing in anger, you had to bring the fueled missile back down to defuel. The first operation with a fueled missile was nearly successful. The drive mechanism held it for all but the last five feet when gravity took over and the missile dropped back down. Very suddenly, the 40-foot diameter silo was altered to about 100-foot diameter. [Ref. 1]*

A radar warning system in Greenland suffered an operational failure in the first month. It reported inbound Russian missiles, but what it actually responded to was...*the rising moon*.

If you make something available to someone, it will at some point be used, even if you didn't intend it to be. For example, there were two cases where North American Radar Defense (NORAD) and Strategic Air Command (SAC) went on alert because radar systems reported incoming missiles. In reality, someone had just loaded a training tape by mistake. After the same incident happened a *second* time, it was finally agreed upon to store the training tapes in a different location! What might have originally been considered human error was actually an error in the *system* which allowed the inevitable human error to happen.

2.2 Findings of the HSE

The English Health and Safety Executive examined 34 accidents that were the direct result of control and safety system failures in a variety of different industries [Ref. 2]. Their findings are summarized in Figure 2-1. Most accidents could have been prevented. The majority of accidents (44%) were due to *incorrect and incomplete specifications*. Specifications consist of both the *functional* specification (i.e., *what* the system should do) and the *integrity* specification (i.e., how *well* it should do it).

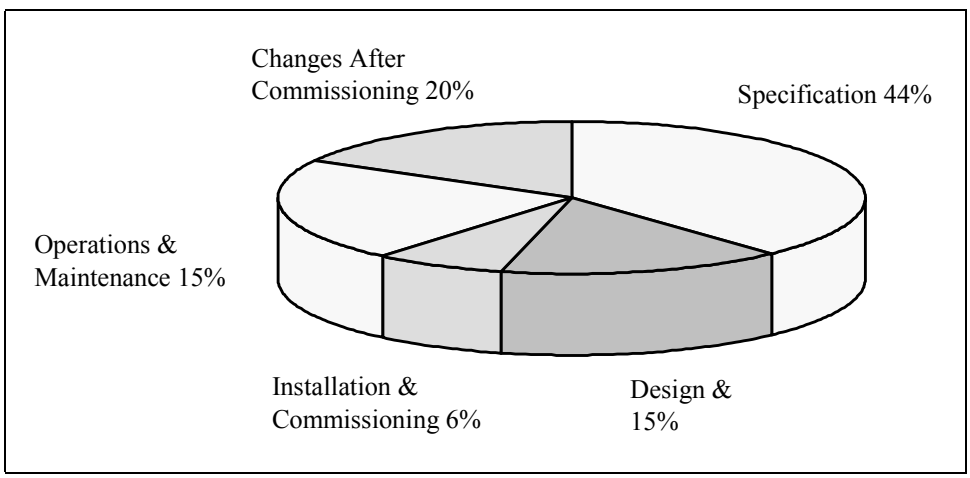


Figure 2-1: Finding of the U.K. HSE, accident cause by phase

There are many examples of functional specification errors. Kletz [Ref. 3] documented a case where a computer controlled an exothermic reactor. When material was added to the reactor the flow of cooling water needed to increase. However, the system was also programmed so that for any

fault in the plant—and many things were categorized as a fault—the outputs would freeze at their last known value. Fate would have it that these two conflicting conditions happened at the same time. Material was added to the reactor and then the system detected a low gear box oil level. The flow of cooling water did not increase so the reactor overheated and discharged its contents. The system did exactly what it was programmed to do. There was no hardware failure.

One Mars probe was designed to circle the planet, yet it crashed into the planet instead. It was determined that the U.S. consortium worked with English units, but the European consortium worked with metric units. There was no hardware failure.

Another Mars probe was designed to land on the planet. Landing legs were designed to deploy at a certain altitude and the rocket motor would then shut off. The mechanical designers knew that the deployment of the landing legs created spurious signals that needed to be temporarily ignored. Unfortunately, this requirement was not documented or passed on to others. The software designers simply programmed the computer to shut off the rocket motor once operation of the landing legs had been detected. The result was that the rocket shut off at too high of an altitude and the satellite was destroyed in a crash landing. There was no hardware failure. A simple omission in the specification resulted in the loss of the satellite which cost over $100 million, not even considering the cost of the booster rocket.

You make shake your head over the silliness of these cases, but there are literally *thousands* of them documented in various texts. There is much to learn from them, even if they happened in an industry different than yours. Trevor Kletz said, "Accidents are not due to lack of knowledge, but failure to use the knowledge we already have." Would you rather learn from the mistakes of others, or make them all yourself, especially if your very life depended on it?

The integrity specification has to do with system performance. The HSE documented a case where a user replaced a relay system with a Programmable Logic Controller (PLC). They implemented the same functional logic and therefore assumed it was the same system—even improved through the use of a newer technology. Unfortunately, the newer technology had completely different failure characteristics than the older system and was actually *less* safe. More will be covered about this topic in subsequent chapters.

Leveson [Ref. 4] has reported that in the vast majority of accidents in which software was involved, flaws could be traced back to the require-

ments—in other words, incomplete or wrong assumptions about the operation of the controlled system or required operation of the computer, unhandled system states, environmental conditions, etc.

Some accidents have happened because systems were used in ways unimagined by the original system designers. For example, one Patriot missile missed an incoming Scud during the first Gulf war. The impact of the Scud at a U.S. facility resulted in multiple fatalities. The Patriot missile system was originally intended and designed for relatively short duration operation. It was not intended to be left on for long periods of time. When operated in such a manner, internal counters and timers differed enough to impact the missile's accuracy. This was the result of the system being continually modified over the years and using different numbering systems in over 20 programming languages. There was no hardware failure.

Needless to say, only the system *user* can develop system specifications. No vendor can possibly imagine or account for all modes of operation. No vendor can tell a user how best to operate their facility or what sort of system logic would be most appropriate.

The next largest portion of problems (20%) found by the HSE were due to changes made after commissioning. What one person thought was a minor, insignificant issue was not properly documented or reviewed by appropriate personnel. Missiles have blown off course and process plants have exploded due to such problems. The end user is primarily responsible for changes made after commissioning.

Operations and maintenance problems were found to be responsible for 15% of accidents. There have been cases where technicians fixed one item, only to introduce a problem elsewhere in the system. In other cases they thought they fixed the item under consideration, yet they actually didn't. End users are primarily responsible for operation and maintenance.

Design and implementation errors accounted for 15% of problems. This is about the only error that falls within the realm of the vendor or system integrator. There have been cases where specifications were correct, but the system supplied did not meet at least one of the requirements and was not thoroughly tested in order to reveal the flaw.

A relatively small percentage of errors (6%) were traced back to installation and commissioning issues.

Overall, similar finding have been published by other organizations, although not necessarily down to the level of detail of actual percentages for each failure classification.

In order to design a safe system, one needs to consider all of these different areas and not just focus on the one that may be the easiest or most convenient to cover. The committees writing the different industry standards and guidelines realized this and attempted to cover all of the bases. This text will attempt to do the same.

2.3 Design Lifecycle

Large systems require a methodical design *process* to prevent important items from falling through cracks. Figure 2-2 shows the lifecycle steps as described in the ANSI/ISA-84.00.01-2004 (IEC 61511 Mod) standard. This should be considered one example only. There are variations of the lifecycle presented in other industry documents. A simplified lifecycle is shown in Figure 2-3. A company may wish to develop their own variation of the lifecycle based on their unique requirements.

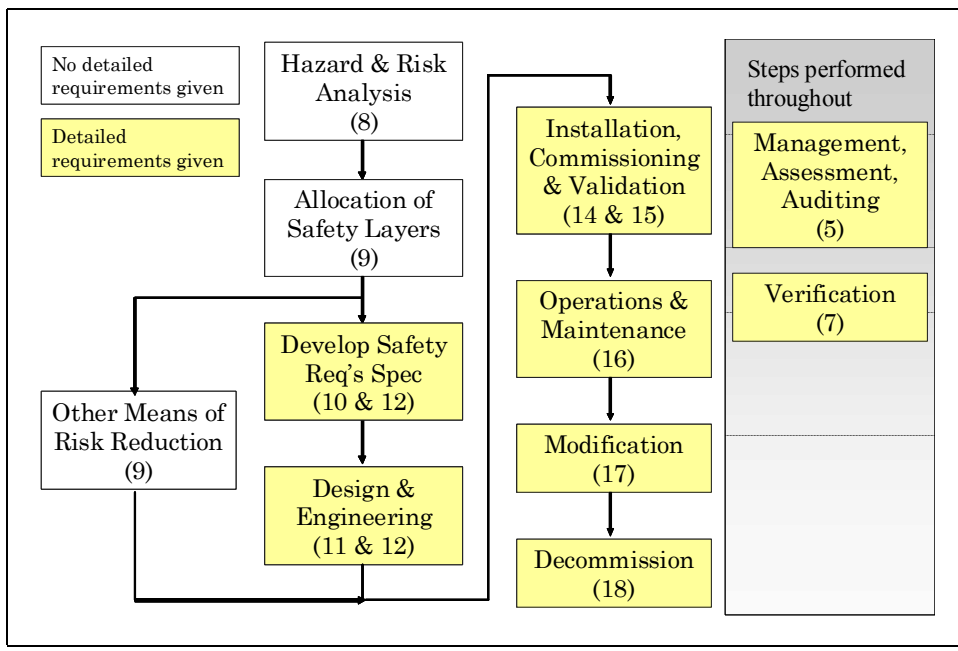


Figure 2-2: SIS Design Lifecycle (with Clause numbers from IEC 61511)

Some will complain that performing all of the steps in the life cycle, like all other tasks designed to lower risk, will increase overall costs and result in lower productivity. One in-depth study in the past [Ref. 4], conducted by a group including major engineering societies, 20 industries, and 60 product groups with a combined exposure of over 50 billion hours, concluded that *production increased as safety increased*. In the U.S., OSHA (Occupational Safety and Health Administration) documented that since the adoption of

their process safety management regulation, the number of accidents has decreased over 20% and companies are reporting that their productivity is *higher*.

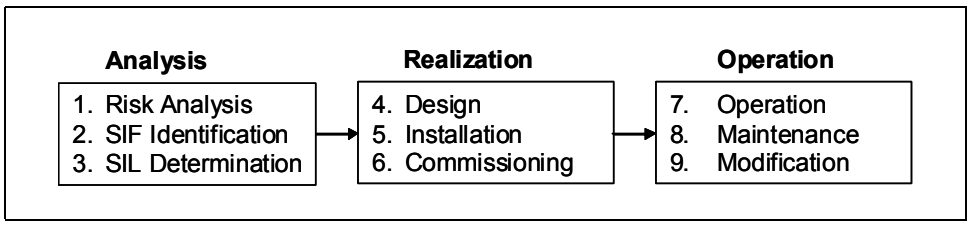


Figure 2-3: Simplified Lifecycle

2.3.1 Hazard & Risk Analysis

One of the goals of process plant design is to have a facility that is inherently safe. As Trevor Kletz said, "What you don't have, can't leak." Hopefully the design of the process can eliminate many of the hazards, such as eliminating unnecessary storage of intermediate products, using safer catalysts, etc. Entire books have been devoted to the subject of inherently safe process designs.

The fist step described in the standard is developing an understanding of the hazards and risks associated with the process. *Hazard analysis* consists of *identifying* the hazards and hazardous events. There are numerous techniques that can be used (e.g., HAZOP, What If, Fault Tree, Checklist, etc.) and many texts to describe each method. *Risk assessment* consists of *ranking* the risk of the hazardous events that have been identified in the hazard analysis. Risk is a function of the frequency or probability of an event, and the severity or consequences of the event. Risks may impact personnel, production, capital equipment, the environment, company image, etc. Risk assessment can either be qualitative or quantitative. Qualitative assessments subjectively rank the risks from low to high. Quantitative assessments, as the name obviously implies, attempt to assign numerical factors to the risk, such as death or accident rates, actual size of a release, etc. These studies are *not* the sole responsibility of the instrument or control system engineer. There are obviously a number of other disciplines required in order to perform these assessments, such as safety, operations, maintenance, process, mechanical design, electrical, etc.

2.3.2 Allocation of Safety Functions to Protective Layers

If the risks associated with a hazardous event can be prevented or mitigated with something other than instrumentation—which is complex, expensive, requires maintenance, and is prone to failure—so much the better. For example, a dike is a simple and reliable device that can easily contain a liquid spill. KISS (Keep it Simple, Stupid) should be an overriding theme.

For all safety functions assigned to instrumentation (i.e., safety instrumented functions), the level of performance required needs to be determined. The standards refer to this as safety integrity level (SIL). This continues to be a difficult step for many organizations. Note that SIL is not directly a measure of process risk, but rather a measure of the safety system performance required in order to control the risks identified earlier to an acceptable level. The standards describe a variety of methods on how this can be done.

2.3.3 Develop Safety Requirements Specification

The next step consists of developing the safety requirements specification. This consists of documenting the input and output (I/O) requirements, functional logic, and the SIL of each safety function. This will naturally vary for each system. There is no general, across the board recommendation that can be made. One simple example might be, "If temperature sensor TT2301 exceeds 410 degrees, then close valves XV5301 and XV5302. This function must respond within 3 seconds and needs to meet SIL 2." It may also be beneficial to list reliability requirements if nuisance trips are a concern. For example, many different systems may be designed to meet SIL 2 requirements, but each will have different nuisance trip performance. Considering the costs associated with lost production downtime, as well as safety concerns, this may be an important issue. Also, one should include *all* operating conditions of the process, from start-up through shutdown, as well as maintenance. One may find that certain logic conditions conflict during different operating modes of the process.

The system will be programmed and tested according to the logic determined during this step. If an error is made here, it will carry through for the rest of the design. It won't matter how redundant or how often the system is manually tested, it simply won't work properly when required. These are referred to as systematic or functional failures. Using diverse redundant systems programmed by different people using different languages and tested by an independent team will *not* help in this situation

because the functional logic they all base their work on could have an error.

A mechanical analogy may be helpful. Suppose we have a requirement for a valve. The valve must be a certain size, must close within a certain period of time, must operate with a certain pressure drop, etc. This represents the functional specification, *what* the valve must do. Different vendors can naturally implement the requirements in different ways. If a carbon steel valve is installed, and two months later the valve is found to have failed due to heavy corrosion, where was the error made? The functional specification just described *what* the valve was to do. In this case the *integrity* specification was incomplete and did not state the corrosive service and that a special material was therefore required. The integrity specification states how *well* the system should perform its function. In terms of an SIS, this would be the SIL.

2.3.4 SIS Design & Engineering

This step covers conceptual design, detail design, and factory testing. Any proposed *conceptual design* (i.e., a proposed implementation) needs to be analyzed to see if it meets the functional and performance requirements. One doesn't pick a certain size jet engine for an aircraft based on intuition. One doesn't size a million dollar compressor by "gut feel". One doesn't determine the size of pilings required for a bridge by trial and error, at least not any more. One needs to initially select a technology, configuration (i.e., architecture), test interval, etc. This pertains to the field devices as well as the logic box. Factors to consider are overall size, budget, complexity, speed of response, communication requirements, interface requirements, method of implementing bypasses, testing, etc. One can then perform a simple quantitative analysis to see if the proposed system meets the performance requirements. The intent is to evaluate the system *before* one specifies the solution. Just as it's better to perform a HAZOP *before* you build the plant rather than afterward, it's better to analyze the proposed safety system *before* you specify, build and install it. The reason for both is simple. It's cheaper, faster and easier to redesign on paper.

Detail design involves the actual documentation and fabrication of the system. Once a design has been chosen the system must be engineered and built following strict and conservative procedures. This is the only realistic method of preventing design and implementation errors that we know of. The process requires thorough documentation which serves as an auditable trail that someone else may follow for independent verification purposes. It's difficult to catch one's own mistakes.

After the system is constructed, the hardware and software should be fully tested at the integrator's facility. Any changes that may be required will be easier to implement at the factory rather than the installation site.

2.3.5 Installation, Commissioning, and Validation

This step is to ensure that the system is installed and started up according to the design requirements and performs per the safety requirements specification. The entire system must be checked, this time including the field devices. There should be detailed installation, commissioning, and testing documents outlining each procedure to be carried out. Completed checks should be signed off in writing, documenting that each and every function has been checked and has satisfactorily passed all tests.

2.3.6 Operations and Maintenance

Not all faults are self-revealing. Therefore, *every* safety instrumented system *must* be periodically tested and maintained. This is necessary in order to make certain that the system will respond properly to an actual demand. The frequency of inspection and testing will have been determined earlier in the lifecycle. All testing must be documented. This will enable an audit to determine if the initial assumptions made during the design (e.g., failure rates, failure modes, test intervals, etc.) are valid based on actual operation.

2.3.7 Modifications

As process conditions change, it may be necessary to make modifications to the safety system. All proposed changes require returning to the appropriate phase of the life cycle in order to review the impact of the change. A change that may be considered minor by one individual may actually have a major impact to the overall process. This can only be determined if the change is documented and thoroughly reviewed by a qualified team. Hindsight has shown that many accidents have been caused by this lack of review. Changes that are made must be thoroughly tested.

2.3.8 Decommissioning

Decommissioning a system entails a review to make sure removing the system from service will not impact the process or any other surrounding units. Means must be available during the decommissioning process to protect the personnel, equipment, and the environment.

Summary

The overall design of safety instrumented systems is not a simple, straightforward matter. The total engineering knowledge and skills required are often beyond that of any single person. An understanding of the process, operations, instrumentation, control systems, and hazard analysis are required. This typically calls for the interaction of a multi-disciplined team.

Experience has shown that a detailed, systematic, methodical, well documented design *process* is necessary in the design of safety instrumented systems. This starts with a safety review of the process, implementation of other safety layers, systematic analysis, as well as detailed documentation and procedures. These steps are described in various regulations, standards, guidelines, and recommended procedures. The steps are referred to as the safety design life cycle. The intent is to leave a documented, auditable trail, and to make sure that nothing is neglected or falls between the inevitable cracks within every organization.

References

1. *Air Force Space Division Handbook.*
2. *Out of Control: Why control systems go wrong and how to prevent failure.* U.K. Health & Safety Executive, 1995.
3. Kletz, Trevor A. *Computer Control and Human Error.* Gulf Publishing Co., 1995.
4. Leveson, Nancy G. *Safeware - System Safety and Computers.* Addison-Wesley, 1995.
5. ANSI/ISA-84.00.01-2004, Parts 1-3 (IEC 61511-1 to 3 Mod). *Functional Safety: Safety Instrumented Systems for the Process Industry Sector.*
6. IEC 61508-1998. *Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems*.
7. *Programmable Electronic Systems in Safety Related Applications, Part 1 - An Introductory Guide.,* U.K. Health & Safety Executive, 1987.
8. *Guidelines for Safe Automation of Chemical Processes.* American Institute of Chemical Engineers - Center for Chemical Process Safety, 1993.
9. Neumann, Peter. G. *Computer Related Risks.* Addison-Wesley, 1995.